

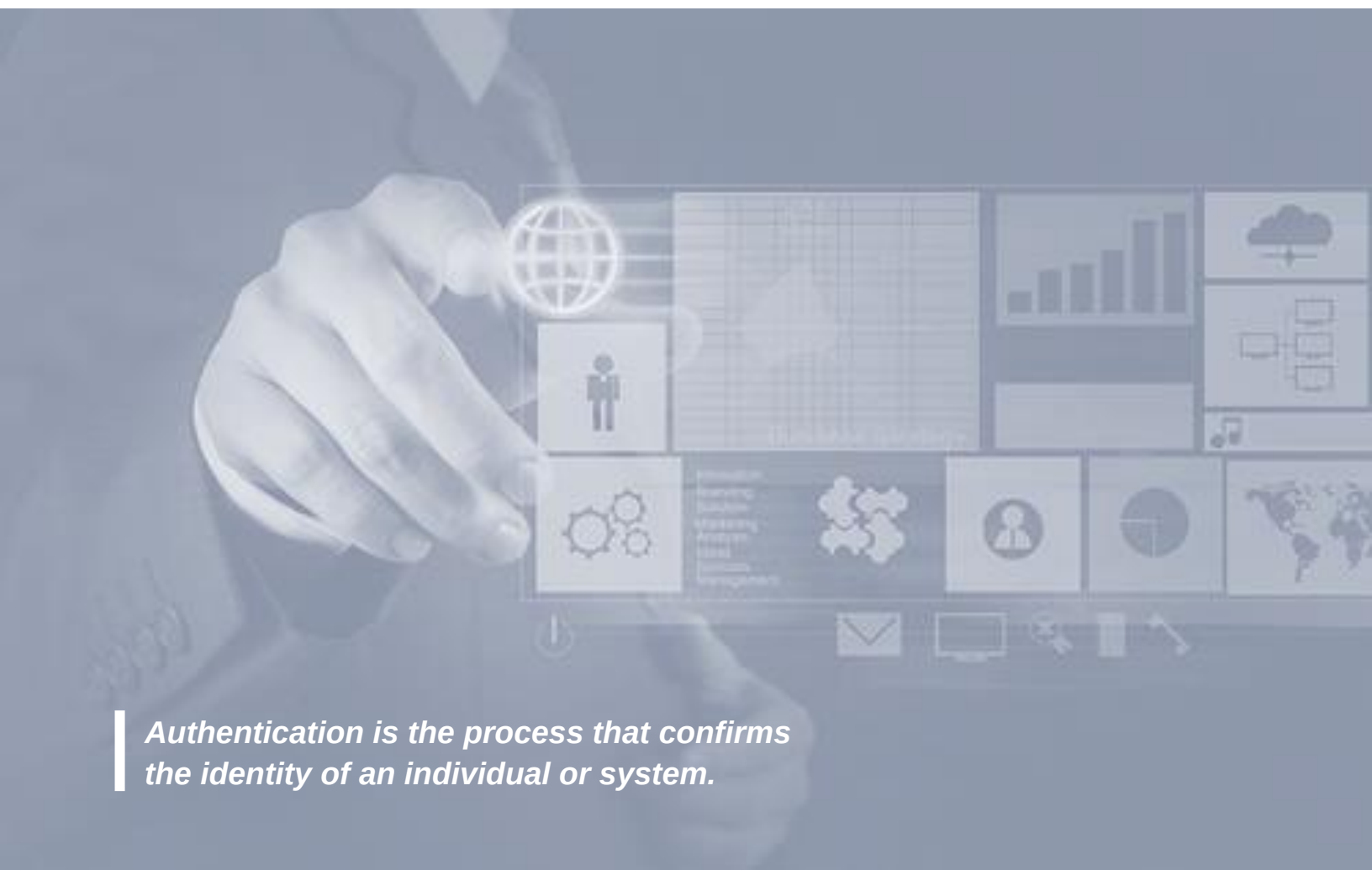
ECOSYSTEM API

Authentication Guide for Service Providers

The *Ecosystem API: Authentication Guide for Service Providers* is a technical document for the Equus Ecosystem. This guide provides an overview of the authentication methods supported by the Ecosystem API for connections to and from the Equus Hub. This guide also includes instructions for setting up OAuth 2.0, X509 certificate, and RSA authentications for each supported authentication method.

The Equus Hub coordinates security and authentication for its inbound and outbound web service requests. To participate in the Ecosystem using the API Method of connection, service providers must expose web service endpoints to the Equus Hub. These endpoints provide connections either directly to the service providers' systems or through a gateway, middleware, or integration platform—providing access to the appropriate data within a service provider's internal systems.

This document is meant to accompany the *Ecosystem API: Connection Guide for Service Providers*, which discusses Ecosystem components, data flows, and the process of connecting service provider systems so that data can be exchanged with AssignmentPro instances.

A background image showing a hand interacting with a digital interface. The interface displays various icons and data visualizations, including a globe, a bar chart, a pie chart, and a world map. The hand is pointing at a globe icon, and another hand is visible at the bottom, interacting with a power button icon.

Authentication is the process that confirms the identity of an individual or system.

CONTENTS

1	Ecosystem Authentication: An Overview	1
2	Equus Hub to Service Provider Authentication	3
2.1	OAuth 2.0 Authentication	3
2.2	X509 Certificate Authentication	4
2.3	RSA Authentication	4
2.4	Basic Authentication	5
3	Service Provider to Equus Hub Authentication	7
3.1	OAuth 2.0 Authentication	7
3.1.1	Obtaining Client Id and Client Secret Credentials	7
3.1.2	Requesting an Access Token	8
3.1.3	Using the Access Token	9
3.2	X509 Certificate Authentication	10
3.3	RSA Authentication	11

EXHIBITS

Exhibit 1:	Authentication support in the Equus Hub	1
Exhibit 2:	Example headers for the RSA authentication method	5
Exhibit 3:	Example headers for the Basic authentication method	6
Exhibit 4:	Screenshot of the Vendor Profile page in the Equus Hub	8
Exhibit 5:	Screenshot of the Vendor Profile page shows per-client OAuth 2.0 credentials	8
Exhibit 6:	Example of valid Client Id and Client Secret responses	9
Exhibit 7:	Example of an invalid Client Secret response	9
Exhibit 8:	Example HTTPS request headers	10
Exhibit 9:	Example client certificate key usage	10
Exhibit 10:	Example C# code showing a method to generate a nonce and signature	12

1 Ecosystem Authentication: An Overview

Authentication is the process that confirms the identity of an individual or system. In the context of Equus Ecosystem communications, all connections must be securely authenticated.

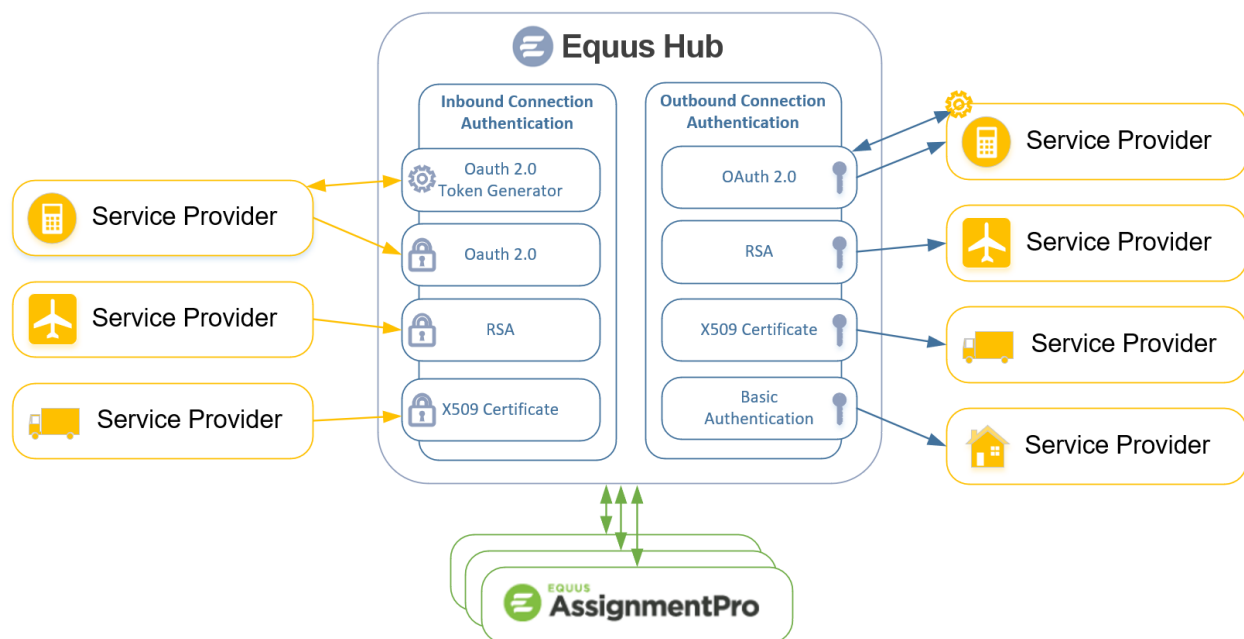
Requests originating in individual AssignmentPro instances, including the initiation of, follow ups to, and cancellations of service orders, flow through the Equus Hub. The Equus Hub then connects to and authenticates with the appropriate service provider endpoint.

Requests originating from a service provider's system, including service status and milestone updates, pass through the Equus Hub. The Equus Hub authenticates the service provider's request before forwarding the request to an AssignmentPro instance.

 See the *Ecosystem API: Connection Guide for Service Providers* for more information about Equus Ecosystem components.

Authentication between AssignmentPro instances and the Equus Hub is handled securely using an Equus proprietary method. The Equus Hub supports industry-standard authentication methods (i.e., OAuth 2.0, RSA, X509, and Basic Authentication), as shown in Exhibit 1, to facilitate communication to and from service provider systems.

Exhibit 1: Authentication support in the Equus Hub



It is also important to note that all traffic to and from the Equus Hub flows across secured HTTPS connections using TLS 1.2 encryption.

Of the authentication methods supported by the Equus Hub for communication to and from service providers' systems, Equus recommends using OAuth 2.0, because of its high level of security and the fact that it is relatively straightforward to troubleshoot issues that may arise.

X509 certificate authentication, which is sometimes called Mutual SSL authentication, is also quite secure. But this method of authentication can be difficult to troubleshoot and requires periodic certificate renewal activities.

RSA security is relatively simple to implement but may be considered less secure than OAuth 2.0 and X509 authentication methods.

Basic authentication is the easiest to use but is also considered the least secure option. For this reason, Basic authentication is not supported for authentication of connections coming into the Equus Hub.



2 Equus Hub to Service Provider Authentication

The Equus Hub can perform authentication with the web service endpoints on service providers' systems using four authentication methods: OAuth 2.0, X509, RSA, and Basic Authentication. Each supported method is described in detail in the following sections, along with setup instructions, header examples, and sample code, where applicable.

2.1 OAuth 2.0 Authentication

OAuth 2.0 is a modern standard for securing access to RESTful web services and APIs. OAuth 2.0 is commonly used in situations where a user would grant permission to a resource holding that user's data. However, it can also be used to facilitate server-to-server authentication such as in Ecosystem transactions, where the credentials exchanged belong to a service account and not to an individual user.

The Equus Hub, when initiating communications with a service provider's system, makes one request to an [Authorization Server](#) to obtain an [Access Token](#). The Equus Hub then provides this token in a second request made to the service provider's web service endpoint (i.e., the [Resource Server](#)).

To enable OAuth 2.0 authentication from the Equus Hub, the service provider will need to provide the following configuration information:

CONFIGURATION INFORMATION	DESCRIPTION
The token request grant type	The Client Credentials grant type is recommended by Equus; however, the Password grant type is also supported.
The Authorization Server URL	This is the URL that the Equus Hub will contact to request the token.
Token request Content-type	Either "application/json" or "application/x-www-form-encoded" content-types are supported.
Token request parameter formatting	Equus needs to know what the Client Id key name and Client Secret key name are and whether they should be transmitted in the HTTPS request header or body.
Token request Audience parameter	This is only required if an Audience parameter should be passed when requesting the token.

CONFIGURATION INFORMATION	DESCRIPTION
Client credentials	These are the values that need to be transmitted for the Client Id and Client Secret .
Username and password	These values are required if the Password grant type is used.
Procedures to retrieve client credentials (if necessary)	If the client credentials and/or username and password need to be retrieved manually, information about that process should also be provided to the Equus team.
Example headers and body of OAuth 2.0 token request	If the service provider has an example of the headers and body of the OAuth 2.0 token request, this would be helpful to the Equus team when configuring this authentication method.

2.2 X509 Certificate Authentication

With X509 certificate-based mutual authentication, both the requester (client) and the server's identity are validated as part of the TLS handshake of an HTTPS request.

To set up this method of authentication, an Ecosystem Integration Analyst will provide the service provider with public certificate authority (CA)-signed X509 certificates for production and non-production environments. The service provider will store these client certificates in their system and associate them with the appropriate Ecosystem-related identities.

When the Equus Hub connects to a service provider's endpoint, it will present the appropriate client certificate, which will be verified and authenticated by the service provider's system.

CA-signed X509 certificates require periodic renewal. Equus will provide replacement certificates when necessary.

2.3 RSA Authentication

This method uses an RSA key pair and nonce value to generate a digital signature that can be authenticated by the service provider's system.

When RSA authentication employed for requests made to service providers' systems, the Equus Hub will include two additional HTTPS request headers, as follows:

nonce	This nonce value is the UTC date and time the request was created, and it is presented in a sortable date/time format.
signature	This is a Base64-encoded string created by the Equus Hub using the private key portion of an RSA key pair to generate a hash of the nonce value.

To configure this authentication method, an Ecosystem Integration Analyst will generate an RSA public/private key pair within the Equus Hub.

- The public portion of the key pair is shared with the service provider, who will use this to validate the digital signature (shown in Exhibit 2) from the Equus Hub.
- The private portion of the key pair remains in the Equus Hub's encrypted database.

When an HTTPS request arrives, the service provider can use the public key and **nonce** to verify that the **signature** value was created using the private key portion of the key pair generated by the Equus Hub.

Exhibit 2: Example headers for the RSA authentication method

```
{
  "content-length": "2158",
  "nonce": "2020-06-11T01:36:32",
  "signature":
    "Yb1lMxaY60Zu7VdOfz+6XeH7jgbdugejtWZISh4XxQ+oI6ju0EUCnqkocYDRZ29T4hO
    ZeCGpKMVTTx3pvKkixBjaNiVXKWt4XxB9tcXMuZvUa0OLu6Go3Pzsszi928pIDCzagh2
    d7SR1U4RHMU72hVWDLfnmLvlGPMblGrHhncCeeJD3FeZUxHHb9iLwXVhGG4RCMh7L9w
    aB7u8GmnyfVcENSmTR77enSHNwW/3HjnMsnxzJAdR0QRaI39GpnwkSCd+N8liXBmTjGb
    KWI8vz2w4l89lKULc6OtzUO2D1SBN9ErSuTY2I09a9banmK+ekEk4bDfchZjdS/UKFXR
    Y4A==",
  "vendorid": "f90871cd-e35d-4a28-b77a-ce37acec1ec0b",
  "assignmentprocompanyid": "1",
  "eventtype": "Initiation",
  "assignmentproinstanceid": "5d39e9b6-7079-42c0-a1a2-8c902c3d0e4d",

  "content-type": "application/json"
}
```

2.4 Basic Authentication

The Basic authentication method is only an option for securing requests made to service providers' systems from the Equus Hub. This method is a simple option that allows authentication with a username and password.

If a service provider's endpoint is set up using the Basic method, the service provider will need to send Equus the credentials with which to authenticate, as shown in the example "authorization" header in Exhibit 3.

While this authentication method is considered less secure, it may be acceptable if additional security measures (such as IP whitelisting, which limits connections between specific internet addresses) are also applied.

Exhibit 3: Example headers for the Basic authentication method

```
{
  "content-length": "2158",
  "authorization": "Basic YWxhc3NpdGVyUmRlbW2wYXNzd29yZA==",
  "vendorid": "f90871cd-e35d-4a28-b77a-ce37acec1ec0b",
  "assignmentprocompanyid": "1",
  "eventtype": "Initiation",
  "assignmentproinstanceid": "5d39e9b6-7079-42c0-a1a2-8c902c3d0e4d",
  "content-type": "application/json"
}
```

3 Service Provider to Equus Hub Authentication

Service providers can authenticate with the Equus Hub using either an OAuth 2.0 token, an RSA signature, or an X509 client certificate. Each supported method is detailed in the following sections, along with setup instructions, header examples, and sample code, where applicable.

3.1 OAuth 2.0 Authentication

Equus recommends that service providers use OAuth 2.0 authentication when connecting to the Equus Hub. Service providers can obtain one or more sets of **Client Secret** values used to request an access token. The access token can be used to authenticate requests.

To authenticate individual HTTPS requests with the Equus Hub, the service provider will request a short-lived access/security token using the **Client Id** and **Client Secret** values. An access token is a data object created by a server, which contains information that is used to authenticate a specific user who is attempting to access a secure system. Once an access token is obtained, it will be sent in a header to authenticate subsequent requests.

3.1.1 Obtaining Client Id and Client Secret Credentials

To begin using this method, the Ecosystem Integration Analyst will provide the service provider with Ecosystem Hub web portal credentials. These credentials allow an individual from the service provider's organization to log into the Equus Hub web portal and retrieve the **Client Id** and **Client Secret** values that are used when requesting OAuth 2.0 authorization tokens.

By default, one set of **Client Id** and **Client Secret** credentials will be used to generate access tokens for requests targeted to any AssignmentPro client instance that the service provider is authorized to communicate with. It is also possible to obtain unique client credentials for each AssignmentPro client instance (as shown in Exhibit 5, but this must be enabled by the Ecosystem Integration Analyst).

The process for obtaining credentials involves the following steps:

- 1) A service provider indicates to the Ecosystem Integration Analyst that OAuth 2.0 authentication will be used for requests made to the Equus Hub.
- 2) The Ecosystem Integration Analyst will create an Equus Hub user account and send the credentials to the service provider.
- 3) The service provider will log into the Equus Hub and navigate to the **Vendor Profile** page as shown in Exhibit 4.
- 4) At the bottom of the **Vendor Profile** page under the **OAuth Credentials** heading, the **Client Id** and **Client Secret** values should be present. If the credentials section is not showing as expected, notify the Ecosystem Integration Analyst who will verify the authentication type configuration.

Exhibit 4: Screenshot of the Vendor Profile page in the Equus Hub

Vendor Profile

Edit Vendor Profile

* Name

ABC Services

Street Address

Phone Number

Company Logo

Innovations_for_Successful_Societies_puzzle_globe_logo.gif

Preview

Remove

Services Provided

Emergency Medical Insurance
Medical Insurance
Moving
Tax Service
Temporary Housing
Visa Service

Country

Search for a Country

Email Address

OAuth Credentials

Client Id

Client Secret

Save

Exhibit 5: Screenshot of the Vendor Profile page shows per-client OAuth 2.0 credentials

OAuth Credentials

Search

Assignment Pro Instance	Client Id	Client Secret
APro Demo New Staging	0a1	45
APro Demo Sales	4e	33
AssignmentProDemoSLIM	2d	01
	9f	7b
	27	
Integration Sandbox	09	98
	df	d2
	14	86
	a2	34

Showing 1 to 9 of 9 entries

3.1.2 Requesting an Access Token

An access token should be obtained and used on every service-provider transaction with an AssignmentPro instance. To do this, the service provider will need to make an HTTP POST request to the token generation endpoint with the appropriate **Client Id** and **Client Secret** values as indicated below:

<https://hub.equusoft.com/OAuth2/Token?clientId=yourClientId&clientSecret=yourClientSecret>

The service provider's actual data will replace the values of **yourClientId** and **yourClientSecret**.

Responses will be returned in JSON format, as shown in Exhibit 6 and Exhibit 7. An access token will remain valid for one hour after it is generated, after which time a new token will need to be obtained.

Exhibit 6: Example of valid Client Id and Client Secret responses

```
{
  "access_token": "3b962490-2f18-4b6b-af12-839b86fa083a",
  "expires_in": 3600
}
```

Exhibit 7: Example of an invalid Client Secret response

```
{
  "error": "invalid client secret",
  "error_description": "Client secret: 1234 did not match stored
secret for vendor with id: 56789"
}
```

3.1.3 Using the Access Token

When an access token is obtained, the service provider should include it as an additional header for subsequent requests to the Equus Hub.

Service providers have two options to send the token obtained from the token generation endpoint (**yourTokenValue**), either will work automatically:

- 1) **Authorization: Bearer yourTokenValue**
- 2) **access_token: yourTokenValue**

When using a single set of credentials, the **AssignmentProInstanceID** of the target instance must be provided in the HTTPS request header along with the access token information, as shown in Exhibit 8.

Exhibit 8: Example HTTPS request headers

```
{
  "authorization": "Bearer c3NpdGVyUmRlbW2wYWxhYXNzd29yZA",
  "vendorid": "f90871cd-e35d-4a28-b77a-ce37acec1ec0b",
  "assignmentprocompanyid": "1",
  "eventtype": "Initiation",
  "assignmentproinstanceid": "5d39e9b6-7079-42c0-a1a2-8c902c3d0e4d",
  "accept": "application/json",
  "content-type": "application/json"
}
```

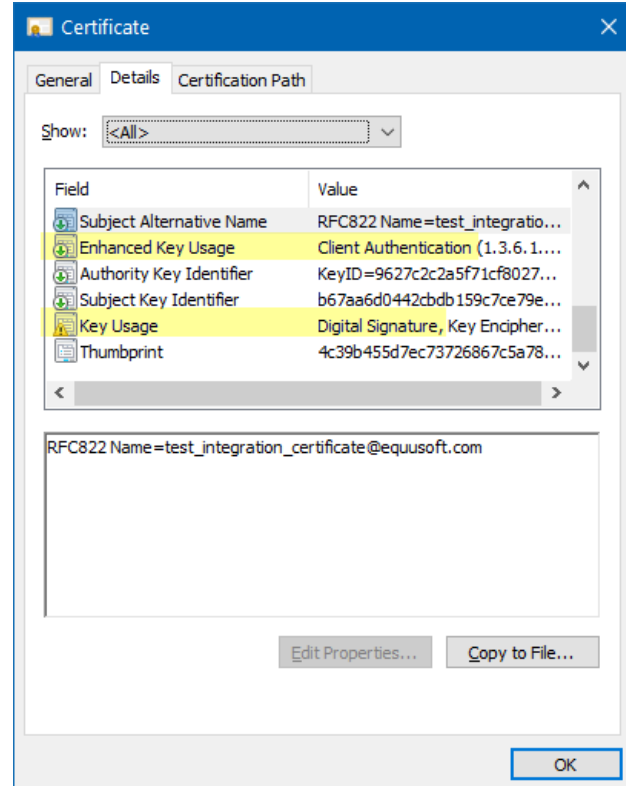
3.2 X509 Certificate Authentication

With X509 certificate-based mutual authentication, both the requester (client) and the server's identity, are validated as part of the TLS handshake of an HTTPS request.

To enable X509 certificate-based mutual authentication for connections made to the Equus Hub, the service provider will need to provide Equus with a valid X509 certificate. The certificate should meet the following requirements:

- ✓ The certificate is signed by a public CA.
- ✓ Each CA certificate in the trust chain is valid.
- ✓ The certificate has an expiration date of at least six months in the future.
- ✓ The certificate has a validity length of at least a year.
- ✓ The certificate file encoding is either binary DER or ASCII/Base64 PEM (i.e., prefixed with a "— BEGIN ..." line) formats.
- ✓ The certificate's private key is not present or otherwise shared with Equus.
- ✓ The certificate should either support all key usages or specifically have the "Digital Signature" key usage and "Client Authentication" enhanced key usage values, as highlighted in Exhibit 9, at right.

Exhibit 9: Example client certificate key usage



For the certificate file to be received via Equus's email system, it should be renamed with a .TXT file extension. Once received, the Ecosystem Integration Analyst will coordinate with an Equus technician to:

- Install the certificate file on the Equus Hub server.
- Associate the certificate's thumbprint with the **Vendor** record in the Equus Hub.

Errors with X509 certificate authentication can be challenging to troubleshoot because invalid or unrecognized certificates are blocked by the web server during the TLS handshake phase of the HTTPS request, where error logging is sparse.

When 401-Unauthorized and 403-Forbidden errors are produced while testing this authentication method, service providers will need to confirm the certificate's validity and thumbprint with Equus. Also, the root and intermediate signing CA certificates may need to be provided if they are not already present on the Equus Hub.

3.3 RSA Authentication

With this method, a service provider's system can use an RSA key pair and nonce value to generate a digital signature that can be authenticated by the Equus Hub.

To set up this type of authentication, the service provider must generate or otherwise obtain an RSA key pair with a key size of at least 2048 bits. The internet describes and references many tools and methods to help with this. Once the key pair is obtained, the service provider would safely store the private key for later use and send the public key to Equus. The Ecosystem Integration Analyst will associate the public key with the service provider in the Equus Hub.

At this point, requests that contain valid information in two additional HTTPS headers will be authenticated. The additional headers are:

noncedate	This nonce value is the UTC date and time the request was created, and it is presented in a sortable date/time format.
signature	This is a Base64-encoded string created by the Equus Hub using the private key portion of an RSA key pair to generate a hash of the noncedate value.

Exhibit 10 is sample C# code showing a method to generate and use a **noncedate** and **signature**.

Exhibit 10: Example C# code showing a method to generate a nonce date and signature

```
/// <summary>
/// Method to build a digital signature nonce for Equus Hub request
authentication
/// </summary>
public class Nonce {
    public Nonce(string privateKey) {
        DateInSortableFormat = (DateTime.UtcNow).ToString("s");

        ASCIIEncoding asciiEncoder = new ASCIIEncoding();
        byte[] dateBinary = asciiEncoder.GetBytes(DateInSortableFormat);

        SHA1Managed hasher = new SHA1Managed();
        byte[] hashedData = hasher.ComputeHash(dateBinary);
        RSACryptoServiceProvider rsaProvider = new
        RSACryptoServiceProvider();

        rsaProvider.FromXmlString(privateKey);
        RSAPKCS1SignatureFormatter rsaFormatter = new
        RSAPKCS1SignatureFormatter(rsaProvider);
        rsaFormatter.SetHashAlgorithm("SHA1");

        DigitalSignature = rsaFormatter.CreateSignature(hashedData);
    }

    // The date to attach to the nonce
    public string DateInSortableFormat {get; private set;}

    // The digital signature to output
    public byte[] DigitalSignature {get; private set;}
}
```

The method shown above might be called from a code snippet like the example shown below.

```
...

    //add the signature and the date that will be used to verify the
    signature
    var nonce = new Nonce("The service provider's private key as xml");
    webClient.Headers.Add("Signature",
    nonce.DigitalSignature.ByteArrayToBase64EncodedString());
    webClient.Headers.Add("NonceDate", nonce.DateInSortableFormat);

...
```

SUMMARY

This guide has provided an overview of the four industry-standard authentication methods supported by the Equus Hub, instructions for setting them up, header examples, and sample code, where applicable.

Rich data, combined with innovative technology tools, helps connect the entire mobility value chain including service-provider systems to facilitate the seamless management of a company's global workforce. Through the Equus Ecosystem, the entire business has visibility into the move status, compliance, costs, and program demographics—all consolidated with performance, KPIs, and employee skills and experiences.

Harness the power of connectivity to simplify the mobility process, gain strategic insights, and enhance the employee experience with the Equus Ecosystem. Global Mobility. Connected.

ABOUT EQUUS

Equus Software is the global leader in cloud-based international relocation and mobility solutions. More than 300 organizations around the world rely on Equus tools and technology to automate mundane, transactional work so that global mobility teams, talent management professionals and other key stakeholders can focus on adding value to the business. Founded in 1999, Equus has a proven track record for delivering cutting-edge talent mobility solutions, continuous innovation and exceptional customer service.



Inquiries@equusoft.com
+1 (303) 292-4200
www.equusoft.com